

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	SHAFFRIA.A	Reg. No.:	192571047
Section / Batch:	B TECH CSBS	Date:	29.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I SHAFFRIA.A(192571047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q75. Debugging Closest Pair Code (Distance Formula Uses Addition Instead of Difference)

The following brute force Closest Pair implementation uses an incorrect distance formula by adding coordinates instead of taking their differences, which produces meaningless geometric distances. Carefully analyze the mathematical expression and explain why the computed minimum is invalid. Apply systematic debugging to restore the correct Euclidean distance formula for all point pairs. Optimize the implementation by storing coordinates in temporary variables for clarity. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
```

```
dist = math.sqrt((points[i][0] + points[j][0])**2 +  
                 (points[i][1] + points[j][1])**2) #
```

ERROR

```
    if dist < min_dist:  
        min_dist = dist  
    return min_dist
```

Q76. Debugging Convex Hull Logic (Endpoint Reversal Not Treated Consistently)

The following brute force Convex Hull helper may produce inconsistent decisions for the same edge when the two endpoints are supplied in reversed order. Carefully analyze how orientation signs change when the segment direction is reversed and explain why the current logic can reject a valid hull edge in one direction but accept it in the other. Debug the code so that edge validation works consistently regardless of endpoint order. Optimize the implementation by structuring the side-checking logic more clearly. Analyze the time complexity of the corrected brute force solution and present the improved logic with proper comments.

```
def is_hull_edge(p1, p2, points):  
    for p in points:  
        val = orientation(p1, p2, p)  
        if val < 0:  
            return False
```

return True

```
main.py
1 import math
2
3 def closest_pair(points):
4     """
5     Finds the minimum Euclidean distance between any two points in a list.
6     """
7     # Initialize minimum distance to infinity
8     min_dist = float('inf')
9     n = len(points)
10
11     for i in range(n):
12         # Optimize by storing coordinates in temporary variables for clarity
13         x1, y1 = points[i]
14
15         for j in range(i + 1, n):
16             x2, y2 = points[j]
17
18             # Corrected Euclidean distance using differences
19             dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
20
21             # Update minimum distance if a closer pair is found
22             if dist < min_dist:
23                 min_dist = dist
24
25     return min_dist
26
27 # Example Execution
28 sample_points = [(1, 2), (3, 5), (6, 7), (2, 3), (8, 1), (5, 6)]
29 print(f"Minimum distance: {closest_pair(sample_points):.2f}")
```

Minimum distance: 1.41

main.py

```
1 def orientation(p1, p2, p):
2     """
3     Standard cross-product to find orientation.
4     Returns >0 for left turn, <0 for right turn, 0 for collinear.
5     """
6     return (p2[0] - p1[0]) * (p[1] - p1[1]) - (p[0] - p1[0]) * (p2[1] - p1[1])
7
8 def is_hull_edge(p1, p2, points):
9     """
10    Checks if the line segment between p1 and p2 forms an edge of the convex hull.
11    Validates consistently regardless of the endpoint ordering.
12    """
13    has_positive = False
14    has_negative = False
15    for p in points:
16        if p == p1 or p == p2:
17            continue
18        val = orientation(p1, p2, p)
19        if val > 0:
20            has_positive = True
21        elif val < 0:
22            has_negative = True
23        if has_positive and has_negative:
24            return False
25    return True
26
27 points_set = [(0,0), (2,5), (5,7), (8,4), (7,0), (4,2), (3,3)]
28 print(f"Edge (0,0)->(2,5) valid? {is_hull_edge((0,0), (2,5), points_set)}")
29 print(f"Edge (2,5)->(0,0) valid? {is_hull_edge((2,5), (0,0), points_set)}")
30 print(f"Edge (4,2)->(3,3) valid? {is_hull_edge((4,2), (3,3), points_set)}")
```



input

```
Edge (2,5)->(0,0) valid? True
Edge (4,2)->(3,3) valid? False
```

Q75. Debugging Closest Pair Code

1. Mathematical Analysis of the Error

- **Invalid Minimum Justification:** The provided code calculates the distance using addition: $(x_1 + x_2)^2 + (y_1 + y_2)^2$. This computes the squared magnitude of the vector sum of the two coordinates, effectively finding the distance from the origin to a new point $(x_1 + x_2, y_1 + y_2)$. This has no mathematical relevance to the actual spatial separation between the two points.
- **Correction:** The correct metric is the Euclidean distance, based on the Pythagorean theorem, which requires taking the difference between the coordinates:
$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

2. Time Complexity Analysis

- **Time Complexity:** The code uses two nested loops. The outer loop runs n times, and the inner loop runs $n - i - 1$ times. The total number of iterations is $\frac{n(n-1)}{2}$. Therefore, the time complexity remains $O(n^2)$, where n is the number of points.

Q76. Debugging Convex Hull Logic

1. Analysis of Orientation Reversal

- **Endpoint Reversal Issue:** The `orientation(p1, p2, p)` function typically uses a cross-product to determine if

point p is to the "left" (positive) or "right" (negative) of the directed line from p_1 to p_2 . If you reverse the endpoints to p_2 to p_1 , the direction of the line flips, which perfectly inverts all the signs of the cross-products.

- **Why the Current Logic Fails:** The original code rejects the edge immediately if it finds *any* negative value (if $val < 0$: return False). This means it only accepts edges where all points lie on the "left". If a perfectly valid boundary edge is passed in reversed order, all other points will lie on its "right" (producing negative values), and the function will incorrectly reject it.

2. Time Complexity Analysis

- **Time Complexity of Helper:** The `is_hull_edge` function iterates through all n points exactly once, making its time complexity $O(n)$.
- **Overall Algorithm Complexity:** In a brute force Convex Hull implementation, this $O(n)$ check is performed for every possible pair of points, which takes $O(n^2)$ time. This makes the overall time complexity of the complete algorithm $O(n^3)$.
